

CS319 Course Project

Deliverable 6 **User and Developer Manual**

Team Members:

Ali Şen / 22203368

Barış Peksak / 22201659

Berkay Şimşek / 22303338

Gökay Nuray / 22302913

Muhammet Furkan Demir / 22302644

Contents

User Manual.....	4
High-Level Description.....	4
Why use CodeArena?.....	4
Installation & Prerequisites.....	4
How to Run the Software.....	5
Using Docker (Backend):.....	5
Manual Startup:.....	5
How to Use the Software.....	6
Accessing the System:.....	6
User Roles & Features:.....	6
How to Report a Bug.....	7
Known Bugs & Limitations.....	8
Comprehensive User Manual.....	9
Upload & Organize (Artifacts).....	9
Create and Manage Studies.....	12
Evaluating Artifacts.....	16
Participant Competency Assessment.....	25
Developer Manual.....	19

User Manual

High-Level Description

CodeArena is a specialized web-based platform designed to facilitate empirical software engineering studies. It serves as a bridge between researchers and participants, streamlining the process of data collection, task evaluation, and skill assessment.

Why use CodeArena?

- For Researchers: It provides a centralized dashboard to create studies, manage artifacts (code files, documents), design quizzes and questionnaires, and track participant progress. It automates the distribution of tasks and collects structured data from evaluations.
- For Participants: It offers a clear, guided interface to view assigned studies, complete qualification quizzes (powered by AI generation), and perform specific coding or evaluation tasks (supporting both blinded and non-blinded modes).
- Integrated Tools: The system features AI-driven quiz generation (via GroqCloud), secure role-based authentication, and automated email notifications.

Installation & Prerequisites

Before installing CodeArena, ensure your system meets the following requirements.

Prerequisites:

1. Java Development Kit (JDK): Version 21 is required for the backend.
 - *Download:* Eclipse Temurin JDK 21
2. PostgreSQL Database: Version 14 or higher.
 - *Download:* PostgreSQL
3. Node.js & npm: Required for the React frontend.
 - *Download:* Node.js (LTS version recommended).
4. Docker (Optional): For containerized deployment.

Configuration Steps:

1. Database Setup

Create a PostgreSQL database named `demodb`. You can do this via the command line or a tool like pgAdmin: *CREATE DATABASE demodb;*

Note: The default configuration assumes a user postgres with password postgres running on localhost:5432.

2. Backend Configuration (backend/src/main/resources/application.properties)

You must configure the application properties file before running the backend.

- Database Credentials: Update spring.datasource.username and spring.datasource.password if your local setup differs from the defaults.
- Groq AI API: The system uses GroqCloud for generating quiz questions.
 - Key: groq.api.key (A default key is provided, but for production, generate your own at [GroqCloud](#)).
 - Model: groq.api.model (Default: llama-3.1-8b-instant).
- Email (SMTP): Configured for Gmail.
 - spring.mail.username: Your sender email.
 - spring.mail.password: Your 16-character App Password (not your login password).
- JWT Security: app.jwt.secret is used for signing tokens. Change this string for production security.

How to Run the Software

You can run the system using Docker (recommended for "production-like" testing) or manually (for development).

Using Docker:

1. Navigate to the root directory.
2. Build the Docker image and Run the container: *docker compose up --build -d*
3. Stop the container: *docker compose down*

Manual Startup:

1. Start the Backend:
 1. Open a terminal in the backend folder.
 2. Run the application using Gradle wrapper:
 - Windows: `.\gradlew.bat bootRun`
 - Mac/Linux: `./gradlew bootRun`
 3. The backend will start on `http://localhost:8080`.

2. Start the Frontend:

1. Open a new terminal in the project root (where `src` and `package.json` reside).
2. Install dependencies: *`npm install`*
3. Start the development server: *`npm run dev`*
4. Open your browser and navigate to the URL shown (usually `http://localhost:5173`).

How to Use the Software

Accessing the System:

- Login: Navigate to `/login`. Use your registered credentials.
- Register: Navigate to `/register` to create a new account. You will be assigned a role (e.g., Participant) upon registration.

User Roles & Features:

Researcher:

- Dashboard: Upon logging in, you will see the Researcher Dashboard. This gives an overview of all your studies, participant progress, and upcoming deadlines.
- Create Study: Click + Create New Study to define a new research project, set access windows, and invite participants.
- Manage Artifacts: Use the Upload Artifacts page to upload code files or documents that participants will evaluate.
- Quizzes:
 - Use Manage Quizzes to create qualification tests.
 - AI Generation: You can auto-generate questions by providing a topic (e.g., "Python Lists"). Note that the free tier limits this to ~5 questions per batch.
- View Submissions: Track completed tasks and view detailed results in View Submissions.

Participant:

- Dashboard: Your hub for all assigned work. It is divided into:
 - Pending Quizzes/Questionnaires: Items you must complete before entering a study.
 - Notifications: Alerts for new invites or approaching deadlines (highlighted in yellow/red based on urgency).

- Assigned Studies: A list of active studies. Click on a study to view its details or start tasks.
- Taking a Quiz: If a study requires a qualification quiz, click the "Take Quiz" button. You must pass the quiz to unlock the study tasks.
- Evaluating Tasks:
 - Navigate to a task via the dashboard.
 - Blinded Mode: Some tasks may hide author information (EvaluationProgressBlindedMode).
 - Submit your evaluation forms or code reviews as instructed on the screen.

Reviewer:

- Reviewer Dashboard: Reviewers have access to a specialized dashboard that provides an overview of all studies in the system.
- Quality Analysis: Reviewers can analyze the quality of participant evaluations across studies, view evaluation metrics, and identify patterns in submission quality.
- Study Access: Reviewers can view all studies (even those they are not directly collaborating on) to perform quality assurance and analysis.
- Export Capabilities: Reviewers can export study data and evaluation metrics for further analysis.

Admin:

- Admin Dashboard: Provides system-wide statistics, user management capabilities, and audit logs to ensure system integrity.

How to Report a Bug

If you encounter an issue, please report it to our issue tracker (e.g., GitHub Issues or Jira). To ensure we can fix the bug quickly, please use the following template:

Bug Report Template:

1. Title: Short summary of the issue (e.g., "Quiz generation fails for topic 'Rust'").
2. Description: What happened? What did you expect to happen?
3. Steps to Reproduce:
 1. Log in as a Researcher.
 2. Go to "Create Quiz".
 3. Enter topic "Rust" and click "Generate".
4. Screenshots: Attach images of the error message or UI glitch.

5. Environment: Browser (Chrome/Firefox), OS (Windows/Mac).
6. Console Logs: Open Developer Tools (F12) -> Console, and copy any red error text.

Known Bugs & Limitations

AI Question Generation Rate Limits:

- The system uses the GroqCloud Free Tier. You may occasionally see a "Rate Limit" error or receive "Fallback Questions" (generic programming questions) if the API is busy or the limit is reached.
- *Workaround:* Wait a few seconds and try generating again.

File Upload Limits:

- Single file uploads are capped at 50MB.
- Total request size is capped at 100MB. Uploading larger datasets will result in an error.

Mobile Responsiveness:

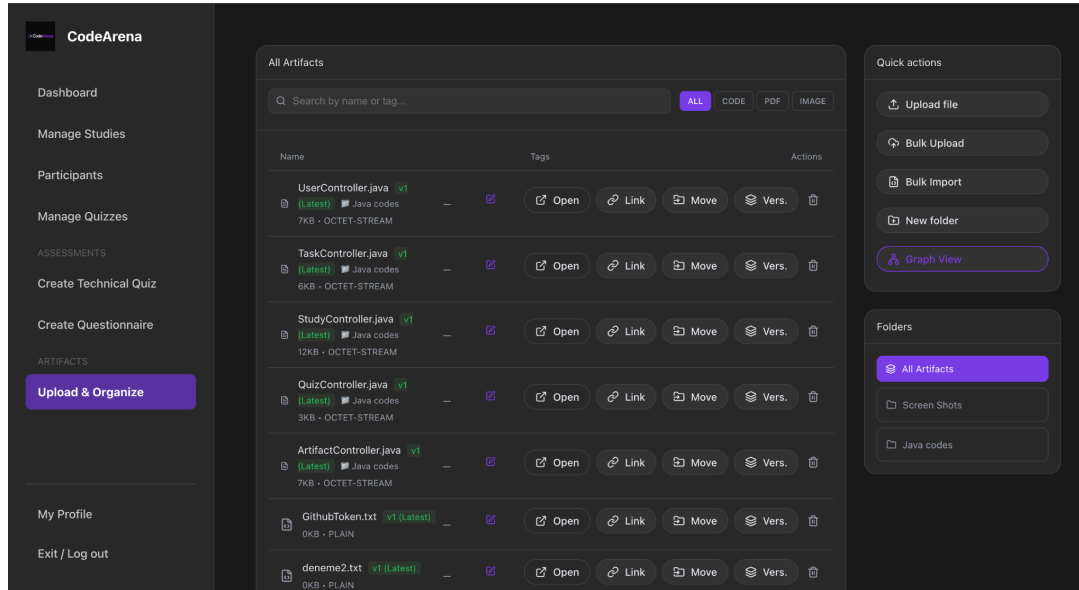
- While the dashboards are functional on mobile, complex code evaluation tasks are optimized for Desktop/Landscape viewing.

Work in Progress:

- The "Forgot Password" functionality requires a properly configured SMTP server. If the email server is down, password resets will not function.

Comprehensive User Manual

Upload & Organize (Artifacts)



Page Overview

The **Upload & Organize (Artifacts)** page is the central workspace for managing all artifacts in the system. It allows users to upload files, organize them using folders and tags, manage versions, and define relationships between artifacts.

The page is divided into three main sections:

- **Artifacts List (Center)**
Displays all uploaded artifacts and serves as the primary interaction area.
- **Quick Actions (Right – Top)**
Provides shortcuts for common actions such as uploading and importing artifacts.
- **Folders Panel (Right – Bottom)**
Enables folder-based navigation and organization of artifacts.

Artifacts List

The Artifacts List shows a clean and simplified view of the repository by displaying only the current (latest) version of each artifact.

List Capabilities

- **Search**
Users can search artifacts by file name or tag.
- **Filtering**
Artifacts can be filtered by type: ALL, CODE, PDF, IMAGE
- **Version Handling**
Older versions are hidden from the main list and can only be accessed via the version drawer.

Row Actions

Each artifact row includes a set of actions that allow users to manage the artifact efficiently:

- **Open**
Opens the artifact for viewing or downloads it, depending on the file type.
- **Edit (Tag Icon)**
Opens the tag editor, allowing users to add, remove, or update artifact tags.
- **Move**
Changes the folder in which the artifact is stored.
- **Link**
Creates logical relationships between the selected artifact and other artifacts.
- **Vers. (Versions)**
Opens a side drawer displaying the full version history of the artifact.
- **Delete (Trash Icon)**
Permanently deletes the artifact and all of its versions from the system.

Uploading Artifacts

The system supports multiple upload methods to accommodate different workflows.

Upload File

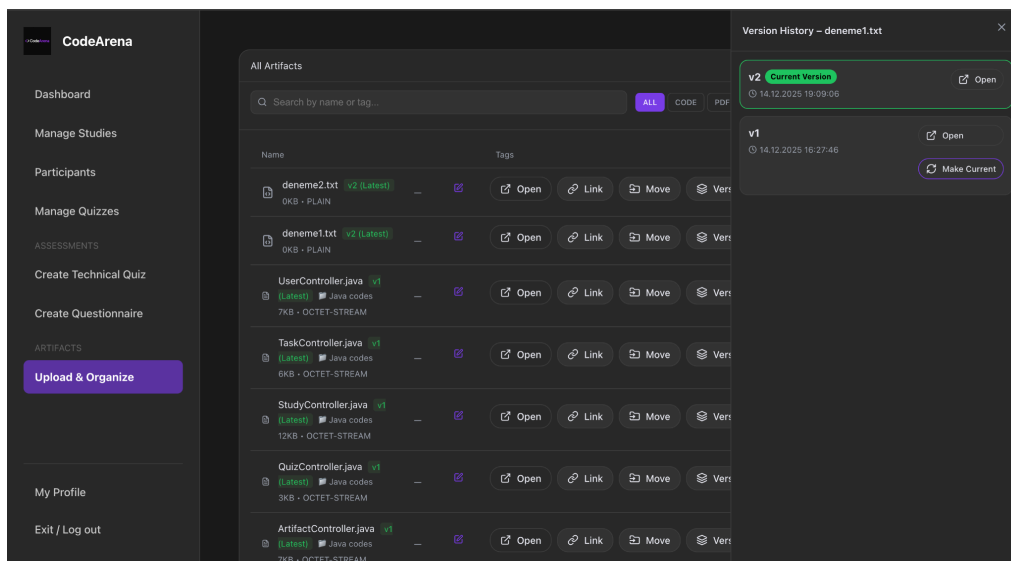
- Upload a single file.
- Tags and a folder can be selected during the upload process.
- The uploaded artifact is automatically created as version v1 and marked as Latest.

Bulk Upload

- Upload multiple files at once directly from the computer.
- Tags and folders are selected once and applied to all uploaded files.
- Each file is stored as a separate artifact with v1 as its initial version.

Bulk Import

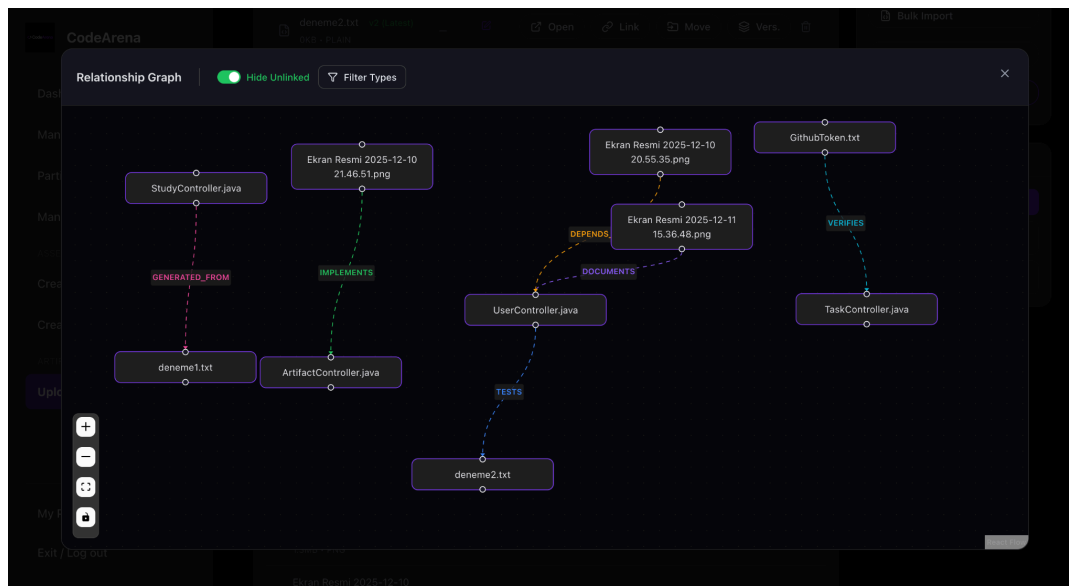
- Upload any number of artifacts using a structured JSON file.
- The JSON file defines artifact metadata such as file content, tags, and folder information.
- All imported artifacts are created as v1 (Latest).



Version Management

Artifact versioning ensures that changes over time are tracked without losing previous states.

- Clicking **Vers.** opens a drawer showing all versions of the selected artifact.
- Only the **current version** is displayed in the Artifacts List.
- All previous versions are read-only.
- Any previous version can be selected and marked as **Make Current**.
- When marked as current, that version becomes Latest and is displayed in the main list.



Linking & Graph View

Artifacts can be logically connected to represent dependencies and relationships.

Linking

- Artifacts can be linked using predefined relationship types such as:
(Implements (Req → Code), Tests (Code → Test), Documents (Doc → Code), etc.)

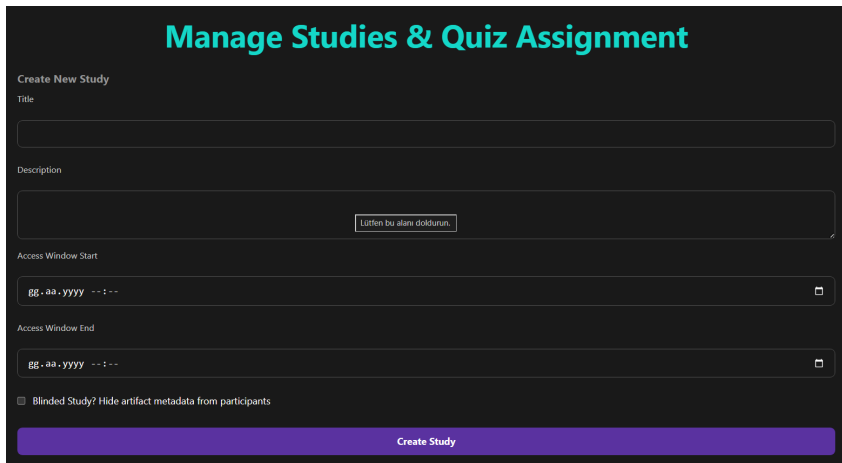
- Linking helps capture semantic and functional relationships between artifacts.

Graph View

- Graph View visualizes artifacts and their relationships as a dependency graph.
- Nodes represent artifacts, and edges represent relationships.
- This view helps users understand structure, dependencies, and impact across the system.

Create and Manage Studies

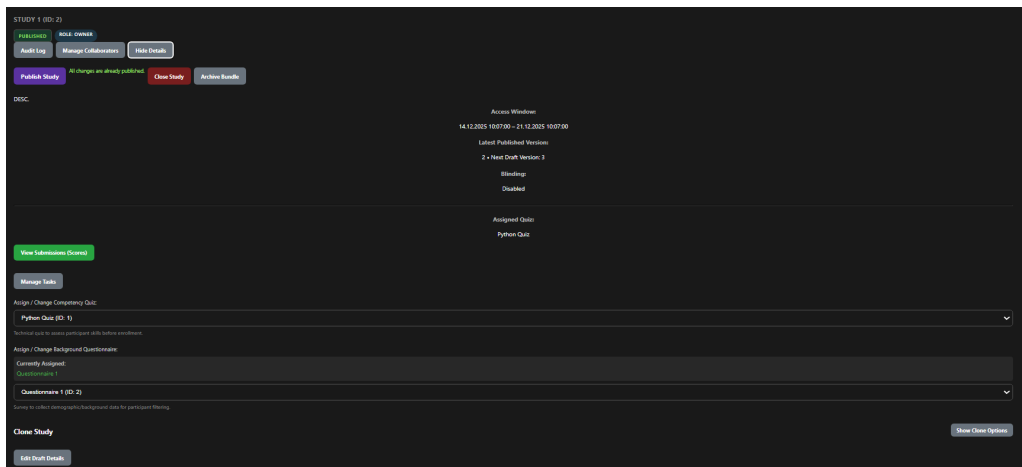
Creating Studies Page



The screenshot shows a dark-themed web interface titled "Manage Studies & Quiz Assignment" in green. Below the title is a section labeled "Create New Study". It contains several input fields: "Title" (a single-line text box), "Description" (a multi-line text box with a "Lütfen bu alanı doldurun." placeholder), "Access Window Start" (a date and time picker showing "gg.aa.yyyy --:--"), and "Access Window End" (another date and time picker). At the bottom, there is a checkbox labeled "Blinded Study? Hide artifact metadata from participants". A large green "Create Study" button is positioned at the bottom right of the form.

- In this page you can set the title, the description, the start and due date for the study. You can also set whether or not the study will be a blinded study or not. When the “Create Study” button is clicked, the study will be created as a draft.

Editing Study Draft Page



- In this page, you can see the details of the study, assign quizzes and questionnaires that the participants will be required to take on the tasks, and also you can clone a study from this existing one.
- You can add new tasks, collaborators, or view submission scores for the quizzes.
- You can also view an audit log for the study that shows all changes that have ever been made in detail.

Audit Log Page

Log Entries (18)			
Timestamp	Action	Actor	Details
14.12.2025 19:10:53	STUDY_PUBLISHED	ResearcherAcc	<pre>{ "versionNumber": 2, "publishedAt": [2025, 12, 14, 19, 10] }</pre>
14.12.2025 19:10:49	STUDY_UPDATED	ResearcherAcc	<pre>{ "fields": ["accessWindowStart", "accessWindowEnd"] }</pre>
14.12.2025 19:10:46	RATING_CRITERION_ADDED	ResearcherAcc	<pre>{ "weight": 1, "name": "CRIT 2", "criterionId": 5 }</pre>
14.12.2025 19:10:15	TASK_ASSIGNED	ResearcherAcc	<pre>{ "artifactId": 2, "artifactBid": 1, "participantId": 1, "taskId": 2 }</pre>

Editing the Study Draft Page

BasicArtifactsCriteriaTasksEligibility

Basic Study Details

Title

STUDY 1

Description

DESC.

Access Window Start

14.12.2025 07:07

Access Window End

21.12.2025 07:07

☐ Blinded Study? Hide artifact metadata from participants

Save Changes

Cancel

Publish Draft

Publish Readiness

Complete the following before publishing draft v3:

There are no changes to publish.

- Prior to publishing, you can edit the basic parameters of the study.
- You can add or remove artifacts from the study.
- You can add criteria to the study that the artifacts will be evaluated by.
- You can add tasks that will be assigned to all participants.
- You can add eligibility rules for the study.

Collaborator Management Page

Study Collaborators

Study ID: 2

Back to Studies

View Study Stats

Invite Collaborator

Select Researcher

Choose Researcher

Or invite by email

researcher@example.com

Role

VIEWER

Invite Collaborator

Current Collaborators (2)

Name	Email	Role	Added	Actions
ResearcherAcc (ID: 2)	mfurkandemir55@gmail.com	OWNER	14.12.2025 19:07:39	Remove
ResearcherAcc2 (ID: 3)	mfurkandemir554223@gmail.com	EDITOR	14.12.2025 19:09:05	Remove

- In this page, you can select a new researcher to collaborate in a study, and assign them whichever role you desire.

Task Management Page

Manage Tasks

STUDY 1 (ID: 2) Status: PUBLISHED (Latest Version: 2)

← Back to Study List

View Study Stats →

Assign New Task

Participant:

— Select a Participant —

Artifact A:

— Select Artifact A —

Artifact B:

— Select Artifact B —

Task Description (Optional):

Enter a description for this task...

Assign Task

Assigned Tasks (2)

Task ID	Participant	Status	Artifact A	Artifact B	Description	Assigned At	Study Version
1	ParticipantAcc (ID: 1)	COMPLETED	robot.js	animation.js	—	14.12.2025 19:09:50	1
2	ParticipantAcc (ID: 1)	IN_PROGRESS	robot.js	animation.js	New task.	14.12.2025 19:10:15	1

- In this page, you can assign new individual tasks to existing participants.

Evaluating Artifacts

Evaluation Page for Participant

Evaluation Progress: Evaluating Code (Task ID: 1)

Code evaluation can be structured around two primary rating criteria: correctness and maintainability. Correctness assesses whether the implementation meets the specified functional requirements, handles edge cases appropriately, and produces accurate outputs under expected and unexpected inputs. Maintainability focuses on how easily the code can be understood, modified, and extended, considering factors such as readability, modularity, naming conventions, and documentation. A balanced evaluation weighs both criteria, recognizing that technically correct code with poor maintainability can increase long-term development costs. Together, these criteria provide a concise yet comprehensive basis for comparing alternative implementations.
Task Note: Evaluate the code according to the given criteria.

Sync
Scroll
Off

Save
&
Exit

Artifact A: quickSort.txt

```
public class QuickSort {
    public static void quickSort(int[] arr) {
        quickSort(arr, 0, arr.length - 1);
    }

    private static void quickSort(int[] arr, int low, int high) {
        if (low < high) {
            int p = partition(arr, low, high);
            quickSort(arr, low, p - 1);
            quickSort(arr, p + 1, high);
        }
    }

    private static int partition(int[] arr, int low, int high) {
        int pivot = arr[high];
        int i = low;

        for (int j = low; j < high; j++) {
            if (arr[j] <= pivot) {
                int temp = arr[j];
                arr[j] = arr[i];
                arr[i] = temp;
                i++;
            }
        }
        int temp = arr[high];
        arr[high] = arr[i];
        arr[i] = temp;
        return i;
    }
}
```

Highlight

Artifact B: MergeSort.txt

```
public class MergeSort {
    public static void mergeSort(int[] arr) {
        if (arr.length < 2) return;
        mergeSort(arr, 0, arr.length - 1);
    }

    private static void mergeSort(int[] arr, int left, int right) {
        if (left >= right) return;

        int mid = (left + right) / 2;
        mergeSort(arr, left, mid);
        mergeSort(arr, mid + 1, right);
        merge(arr, left, mid, right);
    }

    private static void merge(int[] arr, int left, int mid, int right) {
        int n1 = mid - left + 1;
        int n2 = right - mid;

        int[] L = new int[n1];
        int[] R = new int[n2];

        for (int i = 0; i < n1; i++)
            L[i] = arr[left + i];
        for (int j = 0; j < n2; j++)
            R[j] = arr[mid + 1 + j];

        int i = 0, j = 0, k = left;
        while (i < n1 && j < n2) {
            if (L[i] <= R[j]) arr[k] = L[i++];
            else arr[k] = R[j++];
        }
        while (i < n1) arr[k] = L[i++];
        while (j < n2) arr[k] = R[j++];
    }
}
```

Highlight

Artifact C: bubbleSort.txt

```
public class BubbleSort {
    public static void bubbleSort(int[] arr) {
        int n = arr.length;

        for (int i = 0; i < n - 1; i++) {
            boolean swapped = false;

            for (int j = 0; j < n - i - 1; j++) {
                if (arr[j] > arr[j + 1]) {
                    int temp = arr[j];
                    arr[j] = arr[j + 1];
                    arr[j + 1] = temp;
                    swapped = true;
                }
            }

            if (!swapped) break; // already sorted
        }
    }
}
```

Highlight

The image shows a 'Rating Panel' with three sections for different artifacts. Each section contains two horizontal progress bars (one blue, one green) and a text area for a comment. The artifacts are: Artifact A (quickSort.txt), Artifact B (MergeSort.txt), and Artifact C (BubbleSort.txt). At the bottom, there is a blue button labeled 'Submit Evaluation'.

Header / Top Bar

Visuals:

- Page Title: Displays "Evaluation Progress: [Study Title]" along with the unique Task ID for reference.
- Contextual Info: Below the title, it displays the Study Description and any specific Task Notes provided by the researcher to guide the evaluation.
- Status Indicators: If the task is completed or being viewed by an admin, a green [READ ONLY] or [COMPLETED] tag appears.
- Action Buttons (Right Side):
 - Sync Scroll Toggle: A button labeled "Sync Scroll Off/On".
 - Exit Button: Labeled "Save & Exit" (active mode) or "Exit" (read-only mode).

Functionality:

- Navigation: The Exit button saves the current state (if active) and redirects the user back to the Participant Dashboard.
- Scroll Synchronization: Toggling "Sync Scroll" locks the scrollbars of all artifact viewers together. When the user scrolls one document, the others automatically scroll to the same relative percentage, making it easier to compare long files side-by-side.

Artifact Display Area (Middle)

Visuals:

- Grid Layout: The center of the page features a split-screen layout displaying Artifact A and Artifact B side-by-side. If a third artifact exists, the grid adjusts to a three-column layout to include Artifact C.
- Artifact Cards: Each column is contained within a distinct card with:
 - Header: Displays the artifact name (e.g., filename) and the Author's name/avatar. *Note: In Blinded Mode, the author's name and avatar are hidden/anonymized.*
 - Document Viewer: A scrollable window displaying the code or text content of the artifact.
 - Highlight Button: A "Highlight" button located immediately below the document viewer.

Functionality:

- Content Viewing: Renders raw text or code for the participant to read.
- Text Highlighting: Users can select specific lines of text within the viewer and click the "Highlight" button.
 - Annotation: Upon highlighting, the system prompts the user to add an optional comment explaining the highlight.
 - Tooltips: Hovering over an existing highlight displays the attached comment in a floating tooltip.
 - Persistence: Highlights and comments are serialized and saved to the backend, restored automatically if the user leaves and returns.

Rating & Submission Panel (Bottom)

Visuals:

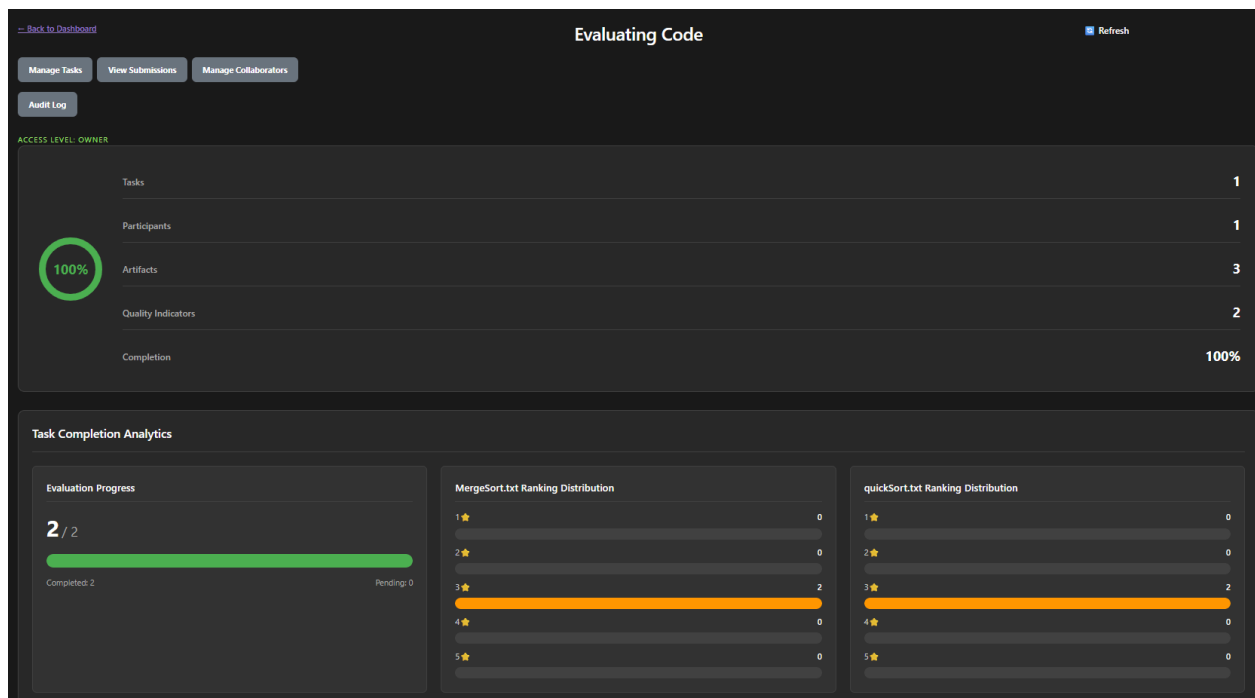
- Sectioned Layout: The bottom panel is divided into blocks corresponding to each artifact (Artifact A, B, and C).
- Rating Sliders: Under each artifact section, the specific Rating Criteria (e.g., "Code Quality", "Readability") defined for the study are listed. Next to each criterion is a slider ranging from 1 to 5.
- Comment Boxes: A large text area is provided for "Overall Comments" regarding that specific artifact.

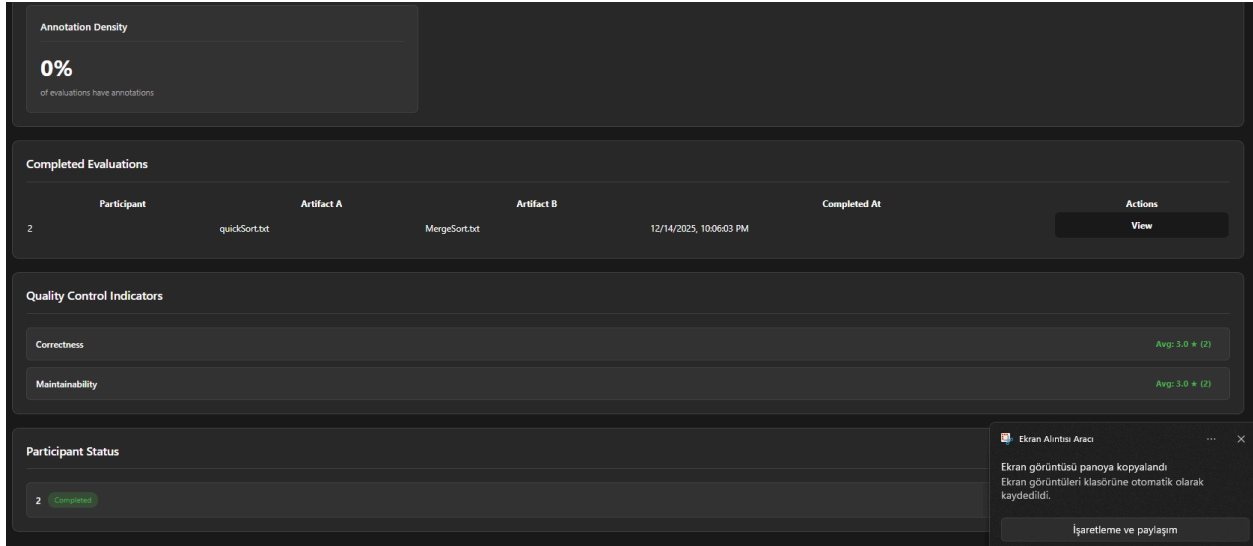
- Status Footer: A "Saving..." indicator and a final "Submit Evaluation" button.

Functionality:

- Scoring: Users drag sliders to assign integer scores (1-5) for each criterion. The system validates that all criteria are scored before allowing submission.
- Auto-Save: The page includes a "debounce" mechanism that automatically saves drafts of ratings, comments, and highlights to the server 1.5 seconds after the user stops typing or clicking, preventing data loss.
- Submission: The "Submit Evaluation" button finalizes the task. It performs a validation check to ensure no ratings are missing, marks the task as COMPLETED in the database, and redirects the user to the dashboard.

Evaluation Page for Researcher





Header & Actions Toolbar

Visuals:

- Navigation: A "← Back to Dashboard" link located at the top left.
- Page Title: The specific Study Title is displayed centrally in large, bold text.
- Refresh Button: A "↺ Refresh" button allows for manual data updates.
- Action Buttons (Role-Dependent): For researchers, a row of quick-action buttons appears:
 - "Manage Tasks"
 - "View Submissions"
 - "Manage Collaborators"
 - "Audit Log"
- Access Level Banner: A small banner indicating the user's role (e.g., "Access Level: OWNER") appears below the header.

Functionality:

- Role-Based Rendering: The available action buttons are dynamically rendered based on the user's permissions (e.g., `canManageTasks`, `canExport`).
- Navigation: The back button intelligently routes the user to the correct dashboard (Admin vs. Researcher) based on their role.
- State Management: The Refresh button increments a `refreshKey` to re-trigger API calls without reloading the entire page.

Analytics Dashboard (Middle Section)

Visuals:

- Study Info Card (Top Row): A wide card featuring:
 - Pie Chart (Left): A visual ring chart showing the overall "Completion Percentage" (Green for >70%, Orange for >40%, Red otherwise).
 - Key Metrics (Right): A vertical list of statistics including Total Tasks, Participant Count, Artifact Count, and Quality Indicators.
- Analytics Grid: A 3-column layout displaying detailed graphical data:
 - Evaluation Progress: A large progress bar showing Completed / Total Evaluations.
 - Artifact Ranking Distribution: Histograms showing the distribution of ratings (1-5 stars) for specific artifacts, helping identify if an artifact is consistently rated poorly or highly.
 - Annotation Density: A statistic card showing the percentage of evaluations that include text highlights or comments.

Functionality:

- Data Aggregation: The system fetches raw task data and calculates completion percentages, variances, and distributions client-side.
- Visual Feedback: Progress bars and charts use color-coding (Green/Orange/Red) to provide immediate visual feedback on the health of the study.

Detailed Lists & Tables (Bottom Section)

Visuals:

- Completed Evaluations Table: A tabular view listing:
 - Participant Name
 - Artifact A & B Filenames
 - Completion Timestamp
 - "View" Button: A button to open the specific evaluation details.
- Quality Control Indicators: A list of the study's rating criteria (e.g., "Code Quality") displaying the Average Rating and total count of ratings received.
- Participant Status List: A list of enrolled participants, displaying:
 - Name
 - Status Badge: Color-coded badges (e.g., "Completed" in Green, "In Progress" in Blue).
 - Task Progress (e.g., "5 / 10 tasks").

Functionality:

- **Blinded Routing:** When clicking the "View" button on an evaluation, the system checks if the study is blinded. If true, it routes to the EvaluationProgressBlindedMode page; otherwise, it opens the standard view.
- **Pagination / Expansion:** The lists (Evaluations, Criteria, Participants) show a limited number of items (default: 5) and include a "Show More" toggle to expand the full list.
- **Real-Time Status:** Participant statuses are derived dynamically from their task completion ratios (e.g., if `completedTasks == totalTasks`, status is "Completed").

Evaluation Page for Reviewer

← Back to Studies

Evaluating Code

Reviewing evaluation quality for this study

58%
Quality Score

1
Total Evaluations

1
Participants

0
Pending Review

Add Comment

Enter your comment about this study...

Submit Comment

Export Statistics

Download study statistics and evaluation metrics in various formats.

JSON CSV ZIP

Individual Evaluations

PARTICIPANT	TASK	TIME SPENT	CONSISTENCY	DETAIL LEVEL	STATUS
2 2	Evaluate the code according to the given criteria.	2 min	75%	Low	Flagged

Header & Navigation Toolbar

Visuals:

- Navigation: A "← Back to Studies" button located at the top left to return to the main study list.
- Page Title: The specific Study Title is displayed prominently in the center.
- Subtitle: A contextual subtitle ("Reviewing evaluation quality for this study") appears below the title to confirm the active mode.
- Access Context: Although not an explicit banner, the view is implicitly secured for the REVIEWER role (as defined in SecurityConfig.java), ensuring only authorized personnel can access these details.

Functionality:

- State Management: Clicking "Back to Studies" clears the selectedStudy state, instantly re-rendering the main dashboard without a page reload.
- Context Loading: Upon mounting or selection, the system triggers fetchStudyEvaluations(study.id) to load specific data for the chosen study.

Analytics Dashboard (Quality Overview Section)

Visuals:

- Quality Overview Card: A dedicated panel featuring high-level quality metrics.
 - Quality Score Ring (Left): A large, dynamic circular chart displaying the overall Quality Score (e.g., "85%"). The ring is color-coded:
 - Green: High Quality ($\geq 85\%$)
 - Amber: Medium Quality (70-84%)
 - Red: Low Quality ($< 70\%$)
 - Breakdown Stats (Right): A horizontal distribution of key counts:
 - Total Evaluations: The aggregate number of completed tasks.
 - Participants: Unique count of users who contributed.
 - Pending Review: The number of tasks explicitly waiting for reviewer verification.

Functionality:

- Dynamic Styling: The getQualityColor helper function dynamically injects CSS variables (e.g., --quality-color) to colorize the score ring and consistency bars based on the numerical value.

- Data Aggregation: Metrics like `qualityScore` and `totalEvaluations` are pre-calculated on the backend or aggregated in the `overallStats` memo hook for immediate display.

Detailed Lists, Actions & Tables (Bottom Section)

Visuals:

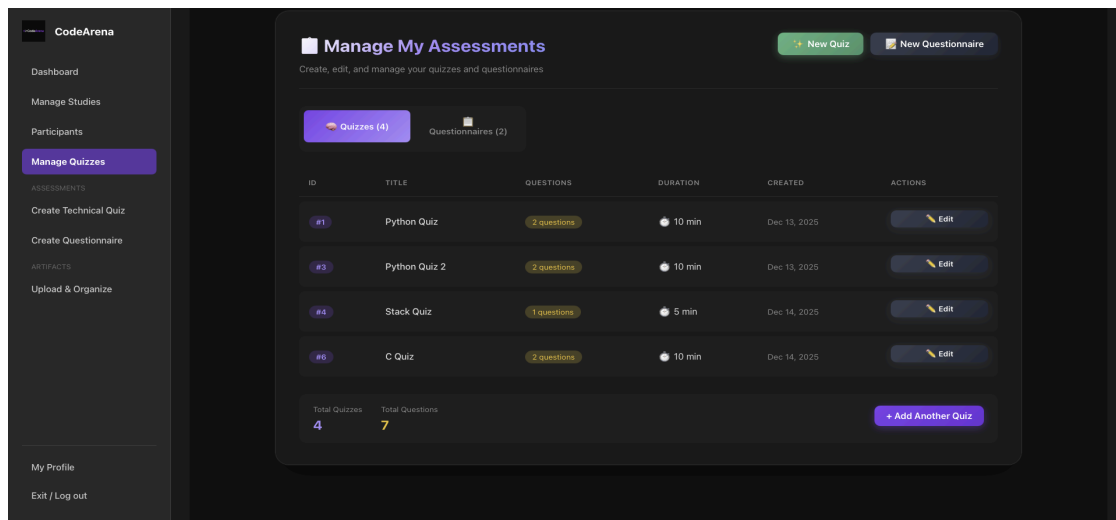
- Reviewer Action Cards: Two side-by-side panels for operational tasks:
 - Add Comment: A text area with a "Submit Comment" button for logging qualitative feedback about the study.
 - Export Statistics: A control panel with three distinct buttons (" JSON", " CSV", " ZIP") for downloading data.
- Evaluations Table: A comprehensive tabular view listing individual submissions:
 - Participant: Displays the participant's name (or ID) with an avatar.
 - Task Title: The specific activity performed.
 - Time Spent: Duration in minutes.
 - Consistency Score: A visual bar chart indicating internal consistency, color-coded to match the quality logic.
 - Detail Level: A badge (e.g., "Low", "High") indicating the depth of the participant's response.
 - Status Badge: A visual indicator of validity:
 -  Flagged: Requires attention.
 -  OK: Passes automated checks.

Functionality:

- Feedback Submission: The "Submit Comment" action performs a POST request to `/api/studies/{id}/reviewer-notes`, providing feedback to researchers.
- Data Export: The export buttons trigger a blob download from the `/api/studies/{id}/export-statistics` endpoint, dynamically handling different file formats (JSON/CSV/ZIP).
- Visual Flagging: The table rows conditionally apply a flagged CSS class to highlight suspicious entries in red/yellow, aiding quick visual auditing.

Participant Competency Assessment

The Participant Competency Assessment module allows researchers to verify that participants possess the necessary technical skills before entering a study. This ensures high-quality data collection by filtering participants based on their quiz performance.



For Researchers: Managing Competency Quizzes

Researchers can create, edit, and manage technical quizzes from the Researcher Dashboard.

Accessing the Quiz Manager

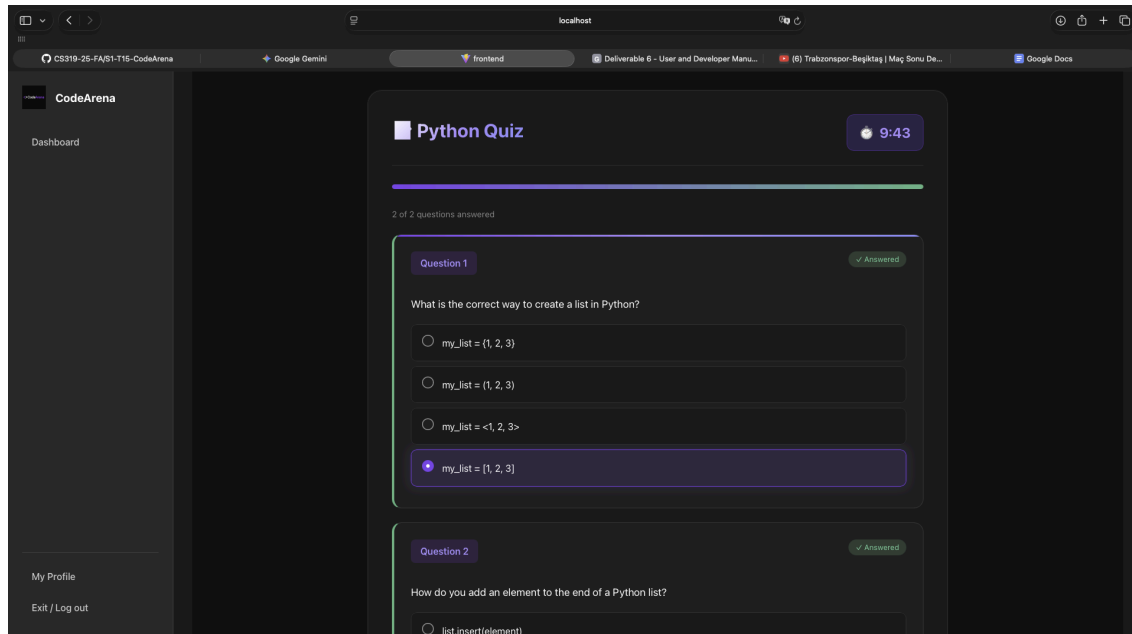
1. Log in to CodeArena as a **Researcher**.
2. Navigate to the **Manage Quizzes** tab in the sidebar.
3. The dashboard displays two tabs:
 - **Quizzes:** For technical competency assessments (graded, timed).
 - **Questionnaires:** For background surveys (ungraded).

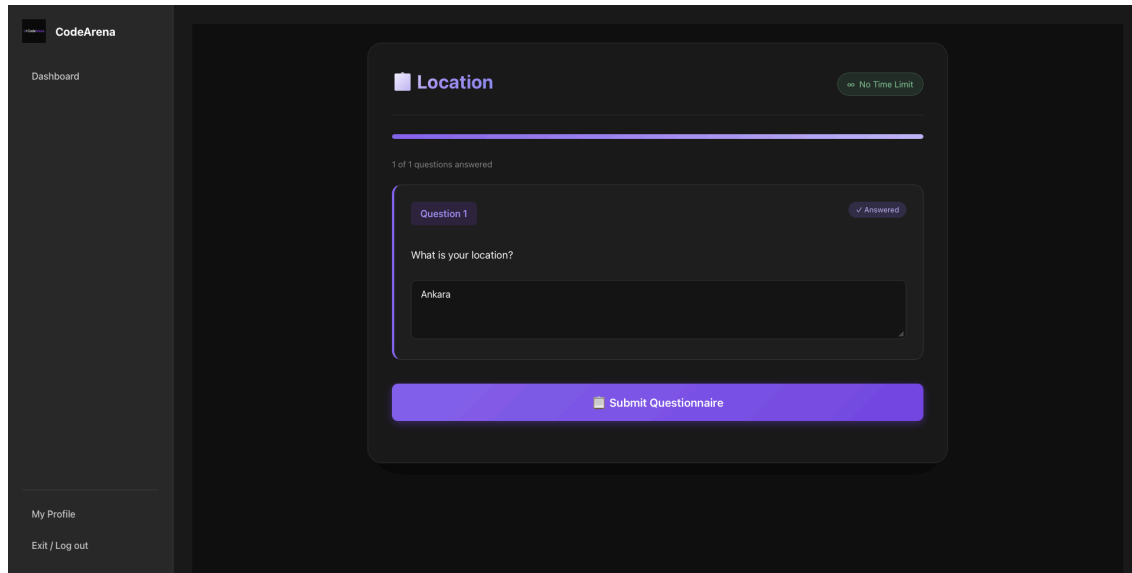
The image displays two screenshots of the CodeArena 'Create Technical Quiz' interface. The top screenshot shows the 'Generate with AI' section, which includes a text input for a topic (e.g., 'Python Lists, React Hooks'), a dropdown for the number of questions (set to 3), and a 'Generate' button. Below this is the 'Quiz Details' section, featuring fields for 'Title' (with placeholder 'Enter quiz title...'), 'Description' (with placeholder 'Describe what this quiz covers...'), and 'Duration (in Minutes)' (with a dropdown set to 10 and a note '0 = unlimited'). The bottom screenshot shows the 'Question 1' section, which includes a 'Question Text' field (with placeholder 'What is the output of the following code?'), a 'Code Snippet (Optional)' field (with placeholder '// Paste code for participants to analyze...'), a 'Question Type' dropdown (set to 'Multiple Choice (Graded)'), and a section for 'Options (Select the correct one)' with two radio button options ('Option 1' and 'Option 2') and an '+ Add Option' button. At the bottom of the form are '+ Add New Question' and '+ Save Quiz' buttons.

Creating a Technical Quiz

1. Click the **"New Quiz"** button.
2. **Basic Details:** Enter the **Title** (e.g., "Java Concurrency Quiz"), **Description**, and **Duration (in minutes)**. Setting duration to **0** makes the quiz untimed.
3. **Adding Questions:** You have two methods to add questions:
 - **Method A: AI Generation (Recommended)**
 - Locate the "Generate with AI" panel at the top of the form.
 - **Topic:** Enter a specific technical topic (e.g., "Python List Comprehensions" or "React Hooks").

- **Count:** Select the number of questions to generate (Max: 5 per batch).
- Click **Generate**. The system uses the Groq/Llama-3 model to automatically create relevant multiple-choice questions with correct answer keys.
- **Method B: Manual Creation**
 - Click "+ Add New Question".
 - Enter the **Question Text**.
 - (Optional) Paste a **Code Snippet** for participants to analyze.
 - **Question Type:** Select "Multiple Choice (Graded)". *Note: "Short Answer" questions are available for surveys but are not automatically graded.*
 - **Options:** Add possible answers and click the radio button next to the **correct option**.
- 4. Click "Save Quiz" to finalize.





For Participants: Taking the Competency Quiz

Participants invited to a study with a prerequisite quiz must pass it to unlock study tasks.

1. **Notification:** Upon logging in, your dashboard will highlight pending quizzes under "Action Required".
2. **Taking the Quiz:**
 - Click "**Take Quiz**".
 - **Timer:** A countdown timer appears in the top right. If the time expires, your answers are submitted automatically.
 - **Progress:** A progress bar at the top tracks how many questions you have answered.
3. **Submission & Results:**
 - Click "**Finish & Submit Quiz**".
 - **Immediate Feedback:** The system instantly calculates your score.
 - **Pass:** If you score **60% or higher**, you gain immediate access to the study tasks.
 - **Fail:** If you score below 60%, you are marked as "QUIZ_FAILED" and cannot proceed to the study tasks unless a retake is authorized.

API Reference

Method	Endpoint	Role	Description
POST	/api/quizzes	Researcher	Creates a new quiz with questions/options.
POST	/api/quizzes/generate	Researcher	Generates questions via AI. Payload: {topic: string, count: int}.
GET	/api/studies/{id}/quiz	Participant	Fetches the quiz assigned to a specific study.
POST	/api/studies/{id}/quiz/submit	Participant	Submits answers. Returns {score: number, passed: boolean}.

Developer Manual

Project Initialization

Obtaining the Source Code:

The project is hosted in a single repository containing both the backend and frontend codebases. To begin, clone the repository using Git:

```
git clone https://github.com/CS319-25-FA/S1-T15-CodeArena.git
cd S1-T15-CodeArena
```

Prerequisites:

Ensure your development environment meets the following specifications:

- Java: JDK 21.
- Node.js: Latest LTS version (for React frontend).
- Database: PostgreSQL 14+.
- Docker: (Optional) For containerized builds.

Directory Structure

The project follows a monorepo-style layout with distinct directories for the backend API and the frontend client.

Root Directory

- backend/: Contains the complete Spring Boot server application.
- src/: Contains the React frontend source code.
- public/ (implied): Static assets for the frontend.

Backend Structure (/backend)

The backend is a standard Gradle-based Spring Boot project.

- build.gradle: Dependency management and build configuration.
- Dockerfile: Multi-stage Docker build configuration.
- src/main/java/com/halenteck/demo/:
 - config/: App configurations (e.g., Database migrations).
 - controller/: REST API endpoints (e.g., StudyController, UserController).
 - dto/: Data Transfer Objects for API requests/responses (e.g., CreateStudyRequest, TaskResponseDTO).
 - entity/: JPA Entities mapping to database tables (e.g., UserEntity, StudyEntity).
 - repository/: Spring Data JPA repositories.
 - service/: Business logic (e.g., GroqService for AI, StudyService).
 - security/: JWT authentication and filter configurations (JwtTokenProvider, SecurityConfig).
- src/main/resources/:
 - application.properties: Main configuration file (DB credentials, API keys).

Frontend Structure (/src)

The frontend is a React application using functional components and Hooks.

- App.jsx: Main entry point and Routing configuration (react-router-dom).
- context/: Global state management (e.g., AuthContext for user sessions).
- components/: Reusable UI components:
 - ProtectedRoute.jsx: HOC for role-based access control.
 - DashboardLayout.jsx: Standard layout for dashboard pages.
- pages/: Full-page views organized by functionality:
 - dashboards/: Role-specific dashboards (ResearcherDashboard, ParticipantDashboard).
 - Login.jsx / Register.jsx: Authentication pages.
- utils/: Utility functions (e.g., permissions logic).

Build & Run Instructions

You can run the system using Docker (recommended for "production-like" testing) or manually (for development).

Using Docker:

4. Navigate to the root directory.
5. Build the Docker image and Run the container: *docker compose up --build -d*
6. Stop the container: *docker compose down*

Manual Startup:

1. Start the Backend:
 4. Open a terminal in the backend folder.
 5. Run the application using Gradle wrapper:
 - Windows: *.\gradlew.bat bootRun*
 - Mac/Linux: *./gradlew bootRun*
 6. The backend will start on <http://localhost:8080>.

2. Start the Frontend:

5. Open a new terminal in the project root (where `src` and `package.json` reside).
6. Install dependencies: *`npm install`*
7. Start the development server: *`npm run dev`*
8. Open your browser and navigate to the URL shown (usually `http://localhost:5173`).

Technical Architecture & Guidelines

Backend Architecture:

- Framework: Spring Boot 3.5.6 with Java 21.
- Security:
 - Stateless authentication using JWT (JSON Web Tokens).
 - Passwords must be hashed (BCrypt is recommended standard with Spring Security).
 - Role-Based Access Control (RBAC) is enforced at the controller level and frontend routes (RESEARCHER, PARTICIPANT, ADMIN, REVIEWER).
- Data Access:
 - Use Spring Data JPA Repositories for database interactions.
 - Dialect: PostgreSQLDialect.
- External Integrations:
 - GroqCloud AI: Implemented in `GroqService.java`. It uses `WebClient` to communicate with the Llama-3 model for quiz generation. It includes retry logic and a fallback mechanism for rate limits.
 - Email: `JavaMailSender` is configured for Gmail SMTP to send notifications.

Frontend Guidelines

- State Management: Use `AuthContext` for user session data. Avoid prop-drilling for global data.
- Routing: All routes are defined in `App.jsx`. Secure routes must be wrapped in `ProtectedRoute` with the `allowedRole` prop.
- Dashboards:

- ResearcherDashboard: Fetches studies and calculates statistics (completion %, participant count) locally after fetching raw data.
- ParticipantDashboard: Aggregates data from multiple endpoints (invites, tasks, quizzes) to present a unified view.

Coding Conventions

- Java:
 - Follow standard Java naming conventions (CamelCase for variables/methods, PascalCase for classes).
 - Use DTOs (e.g., CreateQuestionDTO) for all API payloads; do not expose Entity classes directly in Controllers.
 - Services should handle business logic, while Controllers should handle HTTP mapping and input validation.
- React:
 - Use Functional Components and Hooks.
 - Group related files (component + specific CSS) if necessary, or use the global App.css / Forms.css for shared styles.

Extending the Project

Adding a New User Role:

1. Backend: Update UserRole enum and SecurityConfig to recognize the new role.
2. Frontend: Update ProtectedRoute to allow the new role and add a new Dashboard route in App.jsx.

Adding a New Study Task Type:

1. Backend: Create a new Entity extending the base Task definition. Update StudyController to handle the creation of this specific task type.
2. Frontend: Create a specific evaluation component (e.g., src/pages/TaskEvaluationRouter.jsx) to handle the UI for the new task.

Modifying AI Logic:

To change the AI model or prompt:

1. Edit backend/src/main/resources/application.properties to change groq.api.model.
2. Modify GroqService.java to adjust the buildPrompt method. Ensure the JSON parsing logic matches any changes in the expected output format.

Deliverable Video Link:

<https://drive.google.com/file/d/15n-pbku6mVa0LJByML2KQJZ5P2H1ZBTJ/view?usp=sharing>